

C++26 Execution Control: un'introduzione

Alberto Barbati (Nacon Studio Milan)

C++ Day 2026 – Pavia

Chi sono

Programmo in C++ da quasi trent'anni

Mi occupo principalmente di videogiochi

Mi piace parlare di programmazione e spiegare bene cose complicate


PARALLELISM
CONCURRENCY
PERFORMANCE
SCALABILITY
MODERN C++
C++
C++
<execution>
C++
<execution>
C++
BENVENUTI
NEL FUTURO

Introduzione

C++26 introduce la Execution Library, un framework per la gestione dell'esecuzione asincrona

Questa libreria si pone obbiettivi ambiziosi come:

- introdurre un'astrazione ricca e feconda come furono gli iteratori all'epoca della STL
- essere componibile, estendibile e efficiente
- gestire in maniera unificata qualsiasi "risorsa di esecuzione"

Dalla proposta allo Standard

Nel 2021 viene presentato il paper P2300 "std::execution", che propone il modello Sender/Receiver, perfezionato dall'esperienza sul campo con librerie sviluppate da Facebook/Meta (libunifex) e NVIDIA

A compendio di P2300, NVIDIA ha reso disponibile su Github l'implementazione di riferimento della libreria (https://github.com/NVIDIA/stdexec) che può essere compilata con C++20 ed è tutt'ora mantenuta e aggiornata all'ultima versione delle specifiche

Ci sono anche implementazioni alternative, in particolare https://github.com/bemanproject/execution

Specifica di riferimento

La specifica finale di `<execution>` sarà pubblicata entro breve nello standard C++26

La bozza "finale" di C++26 è il paper N5032 (http://wg21.link/N5032)

La revisione più aggiornata è disponibile su https://eel.is/c++draft/exec


A54
C++26 AND BEYOND
PAVIA
ItalianC++
The Italian C++ Community
C++26
260
km/h
<execution>
parallelism
concurrency
performance
scalaability

Sender

Un *sender* è un oggetto che descrive un'operazione da compiere in maniera potenzialmente asincrona e quali messaggi verranno inviati al suo completamento

La descrizione è *lazy* : l'operazione viene effettivamente eseguita solo quando richiesto esplicitamente

Per lanciare l'esecuzione di un sender si può usare il modo "complicato" oppure le funzioni di utilità ad alto livello che semplificano i casi d'uso più frequenti

Hello, world

```
using namespace ex = std::execution;

int main()
{
    ex::sender auto hello =
        ex::just() | ex::then([] { std::print("Hello, world\n"); });

    std::this_thread::sync_wait(std::move(hello));
}
```

Sender e dove trovarli

Le implementazioni dei sender possono essere:

- Fornite direttamente dalla libreria o da librerie di terze parti
- Ottenute adattando sender esistenti con gli *adapter* della libreria
- Scritti tramite coroutine, usando `ex::task`
- Implementati nativamente tramite tipi che soddisfino i relativi concetti

Completamento

Dopo che l'esecuzione descritta dal sender è partita, ragionevolmente, prima o poi, l'operazione giunge a completamento

Il completamento può accadere immediatamente, nel qual caso diciamo che è *sincrono*, oppure dopo un certo tempo e lo chiamiamo *asincrono*

Quando l'operazione completa, vengono inviati dei "messaggi" su tre canali differenti a seconda dello scenario...



`set_value`

L'operazione può terminare con successo

Il messaggio può veicolare zero o più valori

Un sender può avere più `set_value` con firme differenti



`set_error`

L'operazione può terminare con una condizione di errore

Il messaggio veicola un oggetto che rappresenta l'errore, che solitamente è `std::exception_ptr` ma può essere di qualsiasi altro tipo, ad esempio `std::error_code`

Un sender può avere più `set_error` con firme differenti



`set_stopped`

L'operazione può terminare segnalando di essere stata annullata

Il messaggio non veicola altre informazioni

Questa condizione è usata quando qualcosa (tipicalmente il chiamante stesso) ha esplicitamente richiesto di interrompere l'operazione

Un sender che completa restituendo un valore

```
auto [result] = std::this_thread::sync_wait(
    ex::just(20)
    | ex::then([] (int x) { return x + 1; })
    | ex::then([] (int x) { return x * 2; })
).value();
std::print("The answer is {}\n", result);

// output: The answer is 42
```

Adapter per manipolare i messaggi di completamento

Normalmente, gli adapter propagano tutti i canali di completamento che non gestiscono

Ci sono alcuni adapter specializzati per manipolare i messaggi di completamento:

- `ex::upon_error` e `ex::upon_stopped` eseguono una funzione al ricevemento del messaggio, anziché propagarlo
- con `ex::stopped_as_optional` il messaggio di stop viene rediretto sul canale di successo con un optional vuoto
- con `ex::stopped_as_error` il messaggio di stop viene rediretto sul canale di errore


parallel
par_unseq
par
seq
unseq
Scheduler
Scheduler
Scheduler
Faster, Togheter
C++
<execution>
C++
<execution>
C++
<execution>
C++
<execution>

Scheduler

Uno *scheduler* rappresenta una risorsa di esecuzione: un thread, un pool di thread, un loop di eventi, una risorsa hardware come una GPU, ecc.

Implementare uno scheduler può essere estremamente complicato. È un compito svolto principalmente da library implementor, hardware vendor e system integrator

Applicando la funzione `schedule` sullo scheduler otteniamo un sender che completa la sua esecuzione nell'ambito della risorsa rappresentata dallo scheduler stesso

Esecuzione asincrona

```
ex::scheduler auto sch = ex::get_parallel_scheduler();

std::this_thread::sync_wait(
    ex::schedule(sch)
    | ex::then([] { std::print("Hello, world\n");})
  );
```

Esecuzione parallela

```
void long_task();
void complex_task();

std::this_thread::sync_wait(
    ex::when_all(
        ex::schedule(sch) | ex::then(long_task),
        ex::schedule(sch) | ex::then(complex_task)
    )
);
```

Counting scope

```
ex::counting_scope scope;
auto scope_token = scope.get_token();

ex::spawn(
    ex::schedule(sch) | ex::then(long_task) /* | upon_error */,
    scope_token);
ex::spawn(
    ex::schedule(sch) | ex::then(complex_task /* | upon_error */),
    scope_token);

// l'esecuzione non è bloccata

std::this_thread::sync_wait(scope.join());
```

Esecuzione bulk

```
void update_actor(int x);

std::this_thread::sync_wait(
    ex::schedule(sch) | ex::bulk(ex::par, 100, update_actor)
);
```

Adapter customization

L'implementazione di default di `bulk` esegue semplicamente la funzione in un loop

Tramite tecniche di meta-programmazione a compile-time, il tipo dello scheduler può proporre una implementazione di `bulk` alternativa a quella di default

In questo modo il `parallel_scheduler` è in grado di smistare le singole invocazioni della funzione su più thread

In realtà, qualsiasi adapter può essere customizzato, non solo `bulk` !

Coroutine

L'integrazione della libreria Execution on le coroutine avviene tramite il tipo `ex::task`

Una coroutine di tipo `ex::task` è un sender

All'interno di una coroutine di tipo `ex::task` , ogni sender è *awaitable*

It just works!

Esempio di coroutine

```
ex::task<security_token> authenticate()
{
    for (int try_count = 0; try_count < 3; ++try_count)
    {
        auto username = co_await get_username();
        auto password = co_await get_password();
        if (auto token = co_await get_security_token(username, password);
            token.is_valid())
        {
            co_return token;
        }
        co_await show_message("Invalid username or password, retry");
    }
    throw std::runtime_error("too_many_retries");
}
```

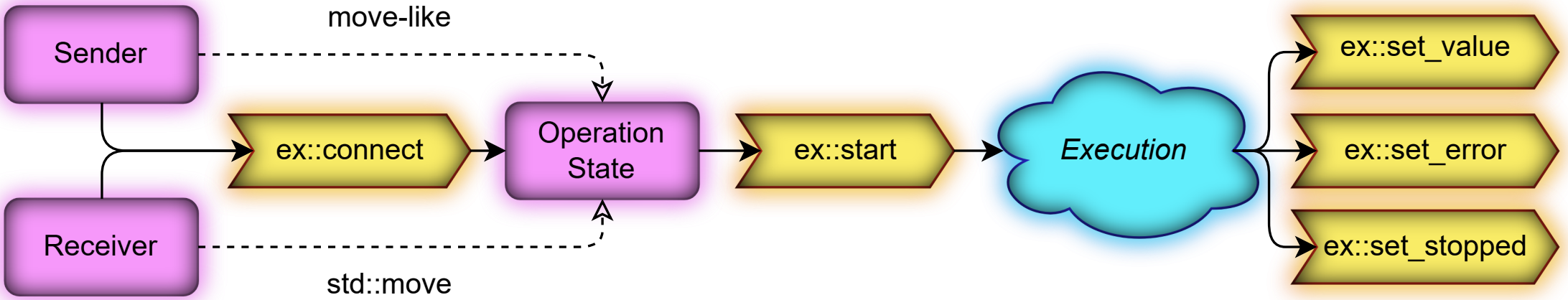

C++
FASTER
SAFER
COOLER
SCHEDULER
THE
POWER
BEHIND
PARALLELISM
SENDER
RECEIVER
C++
PUSH
CONNECT
EXECUTE
std::execution
VALUE
ERROR
STOP
std::execution
C++
IT'S NOT JUST
A LANGUAGE...
IT'S A
POWERPLANT!

Receiver

I *receiver* sono la controparte dei sender, sono oggetti leggeri la cui funzione è di ricevere i messaggi di completamento di un sender e applicarne gli effetti al contesto che ha lanciato il sender

Concettualmente è simile a una callback, con la differenza che un receiver deve gestire tutti i possibili messaggi di completamento

Il motore della libreria



Implementazione di sync_wait (semplificata!)

```
template <ex::sender Snd>
auto simple_sync_wait(Snd&& snd) // -> std::optional<sender_value_t>
{
    using sender_value_t = /* ... */;
    auto local_state = ssw_local_state<sender_value_t>{};
    auto op_state = ex::connect(
        std::forward<Snd>(snd),
        ssw_receiver<sender_value_t>{&local_state});
    ex::start(op_state);
    return local_state.wait();
}
```

Local state

```
template <typename T>
struct ssw_local_state
{
    std::optional<T> value;
    std::exception_ptr error;
    std::atomic_flag complete;

    std::optional<T> wait();
};
```

Local state (wait)

```
template <typename T>
std::optional<T> ssw_local_state<T>::wait()
{
    complete.wait(false); // potenzialmente bloccante
    if (error)
    {
        std::rethrow_exception(error);
    }
    return std::move(value);
}
```

Receiver

```
template <typename T>
struct ssw_receiver
{
    using receiver_concept = ex::receiver_tag;

    ssw_local_state<T>* state;

    void set_value(T value) const noexcept;
    void set_error(std::exception_ptr error) const noexcept;
    void set_stopped() const noexcept;
};
```

Receiver (set_value)

```
template <typename T>
void ssw_receiver<T>::set_value(T value) const noexcept
{
    state->value.emplace(std::move(value));
    state->complete.test_and_set();
    state->complete.notify_one();
}
```

Receiver (set_error)

```
template <typename T>
void ssw_receiver<T>::set_error(std::exception_ptr error) const noexcept
{
    state->error = error;
    state->complete.test_and_set();
    state->complete.notify_one();
}
```

Receiver (set_stopped)

```
template <typename T>
void ssw_receiver<T>::set_stopped() const noexcept
{
    state->complete.test_and_set();
    state->complete.notify_one();
}
```


PROVALA ORA!
C++
<execution>
PARALLELISM
CONCURRENCY
PERFORMANCE
SCALABILITY
MODERN C++
C++
<execution>



Grazie dell'attenzione!

Vi aspetto tutti nella sezione #speaker-corner sul server Discord della Italian C++ Community, per domande, risposte e discussioni

Alberto Barbati (@gamecentric)

Bonus slides: esempio completo di sender nativo

Le slide che seguono mostrano una possibile implementazione di un sender scritto in C++ "nativo" che implementa una lettura da file asincrona, utilizzando l'API Windows ReadFileEx.

L'utilizzo di questo sender dà per scontato che venga gestito dal programma un loop di messaggi che usi una funzione *alertable* come `WaitForMultipleObjectsEx` o `MsgWaitForMultipleObjectsEx`

```
struct readfile_sender
{
    using sender_concept = ex::sender_tag;

    template <typename, typename...>
    static consteval auto get_completion_signatures()
    {
        return ex::completion_signatures<
            ex::set_value_t(std::span<std::byte>),
            ex::set_error_t(std::error_code),
            ex::set_stopped_t()>{};
    }

    HANDLE handle;
    std::span<std::byte> buffer;
    uint64_t offset = 0;

    template <typename Receiver>
    auto connect(Receiver receiver) noexcept;
};
```

```cpp
template <typename Receiver>
struct readfile_opstate : OVERLAPPED
{
    using operation_state_concept = ex::operation_state_tag;

    HANDLE handle;
    std::span<std::byte> buffer;
    Receiver receiver;

    struct try_cancel;
    using stop_callback = /* ... */;
    stop_callback on_stop;

    readfile_opstate(readfile_sender&& snd, Receiver&& rcv) noexcept;

    void start() noexcept;

    static void __stdcall on_complete(
        DWORD errorCode, DWORD numberOfBytesTransfered, OVERLAPPED* overlapped);
};
```

Connect

```
template <typename Receiver>
auto readfile_sender::connect(Receiver receiver) noexcept
{
    return readfile_opstate<Receiver>{std::move(*this), std::move(receiver)};
}
```

Operation state (constructor)

```
template <typename Receiver>
readfile_opstate<Receiver>::readfile_opstate(readfile_sender&& snd,
                                             Receiver&& rcv) noexcept
    : OVERLAPPED{}
    , handle{snd.handle}
    , buffer{snd.buffer}
    , receiver{std::move(rcv)}
    , on_stop{ex::get_stop_token(ex::get_env(receiver)), try_cancel{this}}
{
    // Offset and OffsetHigh are members of a union inside OVERLAPPED
    Offset = static_cast<DWORD>(snd.offset);
    OffsetHigh = static_cast<DWORD>(snd.offset >> 32);
}
```

Operation state (start)

```
template <typename Receiver>
void readfile_opstate<Receiver>::start() noexcept
{
    if (handle == nullptr)
    {
        ex::set_stopped(std::move(receiver));
    }
    else if (!ReadFileEx(handle, buffer.data(), buffer.size(), this, on_complete))
    {
        ex::set_error(
            std::move(receiver),
            make_error_code(::GetLastError()));
    }
}
```

Operation state (try_cancel)

```
template <typename Receiver>
struct readfile_opstate::try_cancel
{
    readfile_opstate* op;

    void operator() noexcept
    {
        CancelIoEx(op->handle.exchange(nullptr), op);
    }
};
```

```cpp
template <typename Receiver>
void __stdcall readfile_opstate<Receiver>::on_complete(
    DWORD errorCode, DWORD numberOfBytesTransfered, OVERLAPPED* overlapped)
{
    auto state = static_cast<readfile_opstate*>(overlapped);
    if (errorCode == 0)
    {
        ex::set_value(
            std::move(state->receiver),
            std::span(state->buffer.data(), numberOfBytesTransfered));
    }
    else if (errorCode == ERROR_OPERATION_ABORTED)
    {
        ex::set_stopped(std::move(state->receiver));
    }
    else
    {
        ex::set_error(std::move(state->receiver), make_error_code(errorCode));
    }
}
```